

Here is the complete, comprehensive transcription of our entire conversation, organized cleanly into structured sections so you can copy and paste it directly into Google Docs.

Distributed Tycho Test Runner Platform Architecture

Target Technologies: HashiCorp Nomad, Go, PostgreSQL, Maven Tycho (OSGi), Docker, Elasticsearch/Vector.

1. Architectural Strategy

To distribute heavy Maven Tycho test suites via Nomad, a monolithic test execution is broken down into parallelized chunks:

- **The Splitter (CI/CD / Orchestrator):** A central coordinator scans the repository for Tycho test plug-ins (pom.xml files with <packaging>eclipse-test-plugin</packaging>).
- **The Workers (Nomad Parameterized Jobs):** The coordinator dynamically dispatches individual Tycho test plug-ins as isolated, cgroup-bounded Nomad tasks.
- **The State Machine & Storage:** A PostgreSQL database manages user queues, priority, fairness, and cluster node states. Workers upload JUnit reports to an artifact store (S3/MinIO) and streams events to Elasticsearch.

2. Worker Node Dockerization (Dockerfile)

Tycho tests often expect an X-server connection because they initialize Eclipse UI plugins. This Docker setup includes a virtual framebuffer (**Xvfb**).

```
FROM maven:3.9-eclipse-temurin-17

# Install Xvfb and web dependencies needed by Eclipse SWT/UI tests
RUN apt-get update && apt-get install -y \
    xvfb \
    libxrender1 \
    libxtst6 \
    libxi6 \
    && rm -rf /var/lib/apt/lists/*

# Wrapper script to start Xvfb before Maven runs
COPY entrypoint.sh /entrypoint.sh
RUN chmod +x /entrypoint.sh
```



```

    status TEXT NOT NULL DEFAULT 'QUEUED',          -- 'QUEUED',
    'RUNNING', 'CANCELLED', 'COMPLETED'
    dispatched_job_id TEXT,
    submitted_at TIMESTAMP WITH TIME ZONE DEFAULT
CURRENT_TIMESTAMP,
    started_at TIMESTAMP WITH TIME ZONE,
    finished_at TIMESTAMP WITH TIME ZONE,
    CONSTRAINT chk_queue_status CHECK (status IN ('QUEUED',
    'RUNNING', 'CANCELLED', 'COMPLETED'))
);

-- Index for optimization during scheduling lookups
CREATE INDEX idx_queue_scheduling ON test_queue (status,
node_pool);

```

4. The Rigid Nomad Job Specification (base-test.nomad.hcl)

This parameterized job configuration maps a single dispatch invocation to a strict cgroup boundary slot. It uses a poststop lifecycle hook container to guarantee immediate artifact or metadata cleanup even if a job is abruptly canceled by a user.

```

job "tycho-distributed-runner" {
  datacenters = ["dc1"]
  type        = "batch"

  # Instantiated dynamically by your Go tool
  node_pool = "default"

  parameterized {
    meta_required = ["TEST_SUITE_MODULE"]
  }

  group "slot-group" {
    count = 1

    # Main Tycho test runner execution
    task "tycho-runner" {
      driver = "docker"

      config {
        image = "my-registry.local/tycho-test-runner:latest"
        args  = ["mvn", "clean", "verify", "-pl",
"$NOMAD_META_TEST_SUITE_MODULE"]

```

```

    }

    # Rigid resource limits defining 1 single slot.
    # Automatically maps to cgroups --cpus / --memory ceilings.
    resources {
        cpu      = 2000 # 2000 mCPU
        memory   = 4096 # 4 GB RAM
    }

    # JVM safe buffer inside the hard 4GB cgroup ceiling
    env {
        MAVEN_OPTS = "-Xmx3072m"
    }
}

# Fires ONLY when "tycho-runner" stops, crashes, or is
# forcefully cancelled
task "cleanup-hook" {
    driver = "docker"

    lifecycle {
        hook      = "poststop"
        sidecar   = false
    }

    config {
        image = "alpine:latest"
        args  = ["sh", "-c", "echo 'Purging rogue workspace files
for ${NOMAD_META_TEST_SUITE_MODULE}...'"]
    }
}
}
}

```

5. Go Orchestrator Core Implementation

This single Go architecture covers the scheduling loop, math-based slot calculation, priority & fair database queuing (SKIP LOCKED), anytime cancellation, live test streaming, and automated updates via Nomad's Event Stream.

```
package main
```

```

import (
    "bufio"
    "context"
    "database/sql"
    "encoding/json"
    "errors"
    "fmt"
    "io"
    "log"
    "os"
    "sync"
    "time"

    "://github.com"
)

const (
    SlotCPU    = 2000
    SlotMemory = 4096
)

type ExecutableTask struct {
    ID             string
    TestSuite      string
    Username       string
    NodePool       string
    DispatchedJobID string
    CancelFunc     context.CancelFunc
}

type TestEvent struct {
    Event string `json:"event"` // "started", "passed", "failed"
    Class string `json:"class"`
    Method string `json:"method"`
    Error string `json:"error,omitempty"`
}

type LiveSuiteProgress struct {
    TotalTests int
    PassCount  int
    FailCount  int
    ActiveTests map[string]string
}

type PlatformOrchestrator struct {
    db      *sql.DB
    nomad   *api.Client
    mu      sync.RWMutex

```

```

    activeTasks    map[string]*ExecutableTask
    activeRuns     map[string]*LiveSuiteProgress
}

func NewPlatformOrchestrator(db *sql.DB, nomad *api.Client)
*PlatformOrchestrator {
    return &PlatformOrchestrator{
        db:          db,
        nomad:       nomad,
        activeTasks: make(map[string]*ExecutableTask),
        activeRuns:  make(map[string]*LiveSuiteProgress),
    }
}

// =====
// 1. SLOT CAPACITY MATHEMATICS
// =====
func (po *PlatformOrchestrator) calculateAvailableSlots(pool
string, selectors map[string]string) int {
    nodes, _, err := po.nomad.Nodes().List(nil)
    if err != nil {
        return 0
    }

    totalAvailableSlots := 0

    for _, nodeStub := range nodes {
        node, _, err := po.nomad.Nodes().Info(nodeStub.ID, nil)
        if err != nil || node.Status != "ready" || node.NodePool
!= pool {
            continue
        }

        matchesSelectors := true
        for k, v := range selectors {
            if node.Meta[k] != v {
                matchesSelectors = false
                break
            }
        }
        if !matchesSelectors {
            continue
        }

        stats, _, err := po.nomad.Nodes().Stats(nodeStub.ID, nil)
        if err != nil {
            continue
        }

```

```

        freeCPU := int(stats.CPUTicksUtil) // Cluster specific
mapping
        freeMemMB := int(stats.MemoryFree / 1024 / 1024)

        slotsByCPU := freeCPU / SlotCPU
        slotsByMem := freeMemMB / SlotMemory

        nodeSlots := slotsByCPU
        if slotsByMem < nodeSlots {
            nodeSlots = slotsByMem
        }

        if nodeSlots > 0 {
            totalAvailableSlots += nodeSlots
        }

    }

    return totalAvailableSlots
}

```

```

// =====
// 2. SCHEDULER WITH WEIGHTED FAIR QUEUING
// =====
func (po *PlatformOrchestrator) FetchNextFairTask(ctx
context.Context, targetPool string) (*ExecutableTask, error) {
    query := `
        WITH user_metrics AS (
            SELECT username, COUNT(*) as running_count
            FROM test_queue
            WHERE status = 'RUNNING'
            GROUP BY username
        )
        SELECT q.id, q.test_suite, q.username, q.node_pool
        FROM test_queue q
        LEFT JOIN user_metrics m ON q.username = m.username
        WHERE q.status = 'QUEUED' AND q.node_pool = $1
        ORDER BY
            (EXTRACT(EPOCH FROM q.submitted_at) * 1000)
            - (q.priority * 10000)
            + (COALESCE(m.running_count, 0) * 5000) ASC
        LIMIT 1
        FOR UPDATE SKIP LOCKED;
    `

    tx, err := po.db.BeginTx(ctx, nil)
    if err != nil {
        return nil, err
    }
}

```

```

    }
    defer tx.Rollback()

    var task ExecutableTask
    err = tx.QueryRowContext(ctx, query, targetPool).Scan(&task.ID,
&task.TestSuite, &task.Username, &task.NodePool)
    if err != nil {
        if errors.Is(err, sql.ErrNoRows) {
            return nil, nil
        }
        return nil, err
    }

    _, err = tx.ExecContext(ctx, `UPDATE test_queue SET status =
'RUNNING', started_at = CURRENT_TIMESTAMP WHERE id = $1`, task.ID)
    if err != nil {
        return nil, err
    }

    if err := tx.Commit(); err != nil {
        return nil, err
    }

    return &task, nil
}

func (po *PlatformOrchestrator) StartSchedulingLoop(ctx
context.Context, pool string, selectors map[string]string) {
    ticker := time.NewTicker(2 * time.Second)
    defer ticker.Stop()

    for {
        select {
        case <-ctx.Done():
            return
        case <-ticker.C:
            if slots := po.calculateAvailableSlots(pool,
selectors); slots == 0 {
                continue
            }

            task, err := po.FetchNextFairTask(ctx, pool)
            if err != nil || task == nil {
                continue
            }

            taskCtx, cancel := context.WithCancel(ctx)
            task.CancelFunc = cancel

```



```

        po.mu.Lock()
        po.activeTasks[task.ID] = task
        po.mu.Unlock()

        go po.executeNomadJob(taskCtx, task)
    }
}

func (po *PlatformOrchestrator) executeNomadJob(ctx
context.Context, task *ExecutableTask) {
    meta := map[string]string{"TEST_SUITE_MODULE": task.TestSuite}
    resp, _, err :=
po.nomad.Jobs().Dispatch("tycho-distributed-runner", meta, nil,
nil)
    if err != nil {
        po.mu.Lock()
        delete(po.activeTasks, task.ID)
        po.mu.Unlock()
        _, _ = po.db.Exec(`UPDATE test_queue SET status =
'QUEUED', started_at = NULL WHERE id = $1`, task.ID)
        return
    }

    po.mu.Lock()
    task.DispatchedJobID = resp.DispatchedJobID
    po.mu.Unlock()

    _, _ = po.db.Exec(`UPDATE test_queue SET dispatched_job_id = $1
WHERE id = $2`, resp.DispatchedJobID, task.ID)

    // Spin off live log/test instrumentation watchers concurrently
    go po.WatchLiveResults(ctx, resp.DispatchedJobID, task.ID)
}

// =====
// 3. ANYTIME CANCELLATION ENGINE
// =====
func (po *PlatformOrchestrator) CancelSuite(taskID string) error {
    po.mu.Lock()
    task, exists := po.activeTasks[taskID]

    // Check database state if it is not residing in active memory
    if !exists {
        po.mu.Unlock()
        var dbStatus, dispatchedID string
        err := po.db.QueryRow(`SELECT status, dispatched_job_id

```

```

FROM test_queue WHERE id = $1`, taskID).Scan(&dbStatus,
&dispatchedID)
    if err != nil {
        return fmt.Errorf("task not found")
    }
    if dbStatus == "QUEUED" {
        _, err = po.db.Exec(`UPDATE test_queue SET status =
'CANCELLED', finished_at = CURRENT_TIMESTAMP WHERE id = $1`,
taskID)
        return err
    }
    if dbStatus == "RUNNING" && dispatchedID != "" {
        _, _ = po.nomad.Jobs().Deregister(dispatchedID,
true, nil)
        _, err = po.db.Exec(`UPDATE test_queue SET status =
'CANCELLED', finished_at = CURRENT_TIMESTAMP WHERE id = $1`,
taskID)
        return err
    }
    return fmt.Errorf("task already completed or cancelled")
}

if task.CancelFunc != nil {
    task.CancelFunc()
}
dispatchedID := task.DispatchedJobID
delete(po.activeTasks, taskID)
po.mu.Unlock()

if dispatchedID != "" {
    _, _ = po.nomad.Jobs().Deregister(dispatchedID, true, nil)
}

_, err := po.db.Exec(`UPDATE test_queue SET status =
'CANCELLED', finished_at = CURRENT_TIMESTAMP WHERE id = $1`,
taskID)
return err
}

// =====
// 4. LIVE LOG SCANNER & TEST RESULTS PARSER
// =====
func (po *PlatformOrchestrator) WatchLiveResults(ctx
context.Context, dispatchedJobID string, taskID string) {
    po.mu.Lock()
    po.activeRuns[taskID] = &LiveSuiteProgress{ActiveTests:
make(map[string]string)}
    po.mu.Unlock()
}

```

```

    // Wait briefly for execution frames to warm up on the target
node
    time.Sleep(2 * time.Second)

    allocs, _, err := po.nomad.Jobs().Allocations(dispatchedJobID,
false, nil)
    if err != nil || len(allocs) == 0 {
        return
    }
    targetAlloc := allocs[0]

    streamCh, errCh := po.nomad.Allocations().FSStream(
        targetAlloc,
        "alloc/logs/tycho-runner.stdout.0",
        "start",
        0,
        ctx.Done(),
        nil,
    )

    pr, pw := io.Pipe()
    go func() {
        defer pw.Close()
        for {
            select {
            case <-ctx.Done():
                return
            case frame := <-streamCh:
                if frame != nil {
                    pw.Write(frame.Data)
                }
            case err := <-errCh:
                if err != io.EOF && err != nil {
                    log.Printf("Nomad stream read error: %v",
err)
                }
            }
        }
    }()

    scanner := bufio.NewScanner(pr)
    for scanner.Scan() {
        line := scanner.Bytes()

        // Concurrently echo standard logging logs down the stdout
line pipe if desired

```

```

        os.Stdout.Write(append(line, '\n'))
    }

    var event TestEvent
    if err := json.Unmarshal(line, &event); err == nil {
        po.updateProgress(taskID, event)
    }
}

func (po *PlatformOrchestrator) updateProgress(taskID string, event
TestEvent) {
    po.mu.Lock()
    defer po.mu.Unlock()

    progress := po.activeRuns[taskID]
    testKey := fmt.Sprintf("%s#%s", event.Class, event.Method)

    switch event.Event {
    case "started":
        progress.TotalTests++
        progress.ActiveTests[testKey] = "RUNNING"
    case "passed":
        progress.PassCount++
        progress.ActiveTests[testKey] = "PASSED"
    case "failed":
        progress.FailCount++
        progress.ActiveTests[testKey] = "FAILED"

        // Pipeline context injection ready for direct bulk index
        ingestion into Elasticsearch
        _ = map[string]interface{}{
            "timestamp": time.Now().UTC().Format(time.RFC3339),
            "task_id":  taskID,
            "class":    event.Class,
            "method":   event.Method,
            "error":    event.Error, // Parsed first-line
signature
        }
    }

    fmt.Printf("\r🔄 Live Updates Task %s -> Total: %d | Pass: %d |
Fail: %d",
        taskID, progress.TotalTests, progress.PassCount,
progress.FailCount)
}

// =====
// 5. AUTOMATED EVENT STREAM LISTENER (CLEANUP)
// =====

```

```

func (po *PlatformOrchestrator) StartEventStreamProcessor(ctx
context.Context) {
    topics := map[api.Topic][]string{api.TopicAllocation: {"*"}}
    eventCh, err := po.nomad.EventStream().Stream(ctx, topics, 0,
nil)
    if err != nil {
        log.Fatalf("Stream hook breakdown: %v", err)
    }

    for {
        select {
        case <-ctx.Done():
            return
        case events := <-eventCh:
            if events.Err != nil {
                continue
            }

            for _, event := range events.Events {
                if event.Topic != api.TopicAllocation ||
event.Type != "AllocationUpdated" {
                    continue
                }

                alloc, err := event.Allocation()
                if err != nil {
                    continue
                }

                taskState, exists :=
alloc.TaskStates["tycho-runner"]
                if !exists || taskState.State != "dead" {
                    continue
                }

                var finalStatus string
                if taskState.Failed {
                    finalStatus = "FAILED"
                } else {
                    finalStatus = "COMPLETED"
                }

                query := `
                    UPDATE test_queue
                    SET status = $1, finished_at =
CURRENT_TIMESTAMP

```

WHERE dispatched_job_id = \$2 AND status = 'RUNNING';

```

_, err = po.db.Exec(query, finalStatus, alloc.JobID)
if err == nil {
    po.mu.Lock()
    // Clear runtime records safely
    for id, t := range po.activeTasks {
        if t.DispatchedJobID == alloc.JobID {
            delete(po.activeTasks, id)
            break
        }
    }
    po.mu.Unlock()
}
}
}
}
}

func main() {
    db, _ := sql.Open("postgres", "postgres://user:pass@localhost/db?sslmode=disable")
    config := api.DefaultConfig()
    nomadClient, _ := api.NewClient(config)

    orchestrator := NewPlatformOrchestrator(db, nomadClient)
    ctx, cancel := context.WithCancel(context.Background())
    defer cancel()

    selectors := map[string]string{"display_type": "xvfb"}
    go orchestrator.StartSchedulingLoop(ctx, "tycho-runners", selectors)
    go orchestrator.StartEventStreamProcessor(ctx)

    select {}
}

```

6. Centralized Logging Infrastructure Alternative (Vector)

If you decide to isolate log collection away from the Go application space entirely, you can run **Vector** as a Nomad `system` job on all nodes. It watches task allocation folders, uses VRL (Vector Remap Language) to filter lines, and writes structured records directly to Elasticsearch.

```

```toml
[sources.nomad_test_logs]
 type = "file"

```

```

include =
["/opt/nomad/data/alloc/*/alloc/logs/tycho-runner.stdout*"]

[transforms.parse_junit_json]
 type = "remap"
 inputs = ["nomad_test_logs"]
 source = ""
 # Coerce string data into structured JSON objects
 parsed, err = parse_json(.message)
 if err == null {
 . = merge(., parsed)
 .is_junit_log = true
 } else {
 .is_junit_log = false
 }
 ""

[sinks.elasticsearch_backend]
 type = "elasticsearch"
 inputs = ["parse_junit_json"]
 endpoint = "http://elasticsearch.local:9200"
 index = "tycho-regressions-%Y.%m.%d"

```

---

## 7. Extending to Test-Level Granularity Strategy

Shifting from distributing whole plugin modules (**suite-level**) to running individual Java test methods (**test-level**) inside single Nomad tasks hits an architectural bottleneck: **Eclipse OSGi Boot Overhead**. It takes 15–45 seconds for a Tycho container to fully load its platform dependencies before running any test.

To resolve this efficiently, implement a **Master-Worker Agent model** instead of generating raw Nomad tasks for single methods:

1. Use the Go orchestrator to scale up long-running **Worker Containers** via Nomad.
2. The Tycho worker container stays fully booted and runs an internal listener loop.
3. The worker fetches single test method assignments from an execution queue hosted in your Go backend (via gRPC/TCP), executes them inside the running Eclipse framework using a custom JUnit runner class, returns pass/fail state immediately, and transitions to the next item without restarting the underlying container context.

---

Feel free to paste this straight into your project's repository documentation or architecture boards! If you need to make any design adjustments to your Go backend, let me know.